iBarangay: BARANGAY DOCUMENT REQUEST SYSTEM

PROJECT DEVELOPMENT DOCUMENTATION

Group Number: 12
Course / Section: BSIT 3-1
Instructor / Adviser: Mr. John Kenneth Timblaco
Documentation Date: October 25, 2026

Group Members and Roles

Name	Assigned Role
Francisco, Helix	Project Manager
Amoranto, Denielle	UI/UX
Amoranto, Denielle	Front-end
Francisco, Helix	Back-end
Francisco, Helix; Amoranto, Denielle	Quality Assurance

Project Repository: [helixfrancisco92-eng/Baranggay-Application: A Barangay Management and Citizen Services Portal]

Project Tracker: [BSIT 3-1] - PROJECT TRACKER.xlsx

---

1. SYSTEM OVERVIEW

1.1 Project Introduction

Traditional barangay processes often rely on manual procedures for managing resident information, processing requests, handling concerns, and communicating important announcements. These processes can require significant time and effort from both barangay officials and citizens, while also making information more difficult to organize, track, and retrieve efficiently.

To address these challenges, the developers proposed **iBarangay: Barangay Document Request System**, a centralized digital platform designed to streamline barangay-related processes and information management. The system provides an organized way of registering residents, accepting and tracking document requests, managing incident reports, and publishing community announcements.

Through digital workflow automation and robust database management, the system aims to reduce citizen inconvenience while helping barangay officials organize, monitor, and process transactions efficiently.

1.2 Purpose of the Project

The purpose of iBarangay is to develop an automated database application that improves the delivery and tracking of barangay services. It aims to eliminate heavy reliance on paper-based transactions, minimize clerical errors, and provide real-time updates regarding service requests.

Specifically, the system is designed to:

1. Provide a centralized digital repository for resident records and demographic profiles.
2. Facilitate online submission, tracking, and processing of document requests and incident reports.
3. Automatically compute document issuance fees and track processing status from submission to release.
4. Maintain a clear audit trail of all request status transitions and administrative approvals.
5. Securely govern user operations using role-based access controls for citizens, staff, and administrators.

1.3 Project Scope

The scope of iBarangay focuses on digital management and processing of common barangay-level transactions. It encompasses resident registration, document request processing, requirement mapping, status auditing, and official announcements.

The system incorporates database operations that allow authorized administrators and staff to create, retrieve, update, and manage records while permitting citizens to log requests and view status updates.

The system is strictly limited to barangay-level services and records. It does not handle municipal, city, or national government agency workflows, nor does it replace the decision-making authority of local barangay officials.

1.4 Main System Functions

Function	Description
Resident Management	Handles citizen registration, profile management, and resident verification.
Document Request Processing	Enables citizens to request barangay clearances, certificates, and permits.
Requirement Mapping	Maps mandatory document requirements dynamically per document type.
Complaints & Incident Logging	Records and tracks citizen grievances and community incident reports.
Transaction Tracking & History	Logs real-time status updates and provides complete processing history.
Announcements Management	Allows administrators to publish community updates and public notices.
Role-Based User Management	Restricts administrative operations based on system user roles.

---

2. USER ROLES

This section identifies the intended users of the application, their responsibilities, and their access level to system data.

2.1 Application User Roles

User Role	Responsibilities	Data Access
Barangay Officials / Administrators	Manages system settings, staff privileges, document types, requirements, announcements, and high-level approvals.	Full access to create, view, update, and modify all system data, audit logs, and user roles.
Barangay Staff	Processes resident document requests, verifies uploaded requirements, updates transaction statuses, and handles incident records.	Access to view resident details, update request statuses, log administrative remarks, and add status history records.

User Role	Responsibilities	Data Access
Barangay Citizens / Residents	Registers user accounts, submits document requests, uploads prerequisites, files complaints, and monitors transaction progress.	Access restricted to viewing personal profile data, submitted requests, public announcements, and personal status updates.

---

3. DEVELOPMENT PROGRESS LOG

This section documents the development activities performed by the team, including project phases, member responsibilities, and commit logs.

3.1 Development Progress

Development Activity	Status / Progress
Project Planning	Completed 100%
Requirements Gathering	Completed 100%
Database Design	Completed 100%
Relational Schema & Normalization	Completed 100%
PostgreSQL Environment Verification	Completed 100%
Database DDL Execution & Constraints	Completed 100%
User Interface & Front-end Development	Completed 100%
Back-end & API Integration	Completed 100%
Quality Assurance & Testing	Completed 100%
Project Documentation	Completed 100%

3.2 Development Log

Development Activity	Responsible Member/s	Status
Overall project planning, resource mapping, and timeline oversight	Francisco, Helix	Completed

Development Activity	Responsible Member/s	Status
Requirements gathering, data identification, and business rules formulation	Amoranto, Denielle; Francisco, Helix	Completed
Conceptual ERD modeling, relational schema design, and normalization	Amoranto, Denielle; Francisco, Helix	Completed
PostgreSQL database creation, environment setup, and script execution	Francisco, Helix	Completed
User interface wireframing, layout design, and client-side screens	Amoranto, Denielle	Completed
Application API, business logic, and PostgreSQL database integration	Francisco, Helix	Completed
Test case creation, system flow verification, and issue tracking	Francisco, Helix; Amoranto, Denielle	Completed
Technical documentation, database dictionary, and progress reporting	Amoranto, Denielle;Francisco, Helix	Completed

3.3 Repository and Commit History

The group utilizes Git and a centralized repository to track codebase changes, database migrations, and structural documentation.

Key Commits:

- Initial commit: Project structure and documentation blueprints
- Database: Created DDL scripts for 7 core PostgreSQL tables
- Database: Applied PK, FK, CHECK, UNIQUE, and DEFAULT constraints
- UI/UX: Added front-end dashboard and document request screens
- Back-end: Implemented database connection routines and CRUD logic

---

4. DATABASE IMPLEMENTATION

4.1 Database Design

The database design phase focuses on establishing a normalized, consistent PostgreSQL schema to store resident profiles, document categories, requirements, requests, and transaction histories.

- **Database Name:** PPGG_projectdb
- **Number of Tables:** 7
- **Primary Keys Established:** Yes
- **Foreign Keys Established:** Yes
- **Integrity Constraints Implemented:** PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, DEFAULT

4.1.1 Entity Relationship Diagram (ERD)

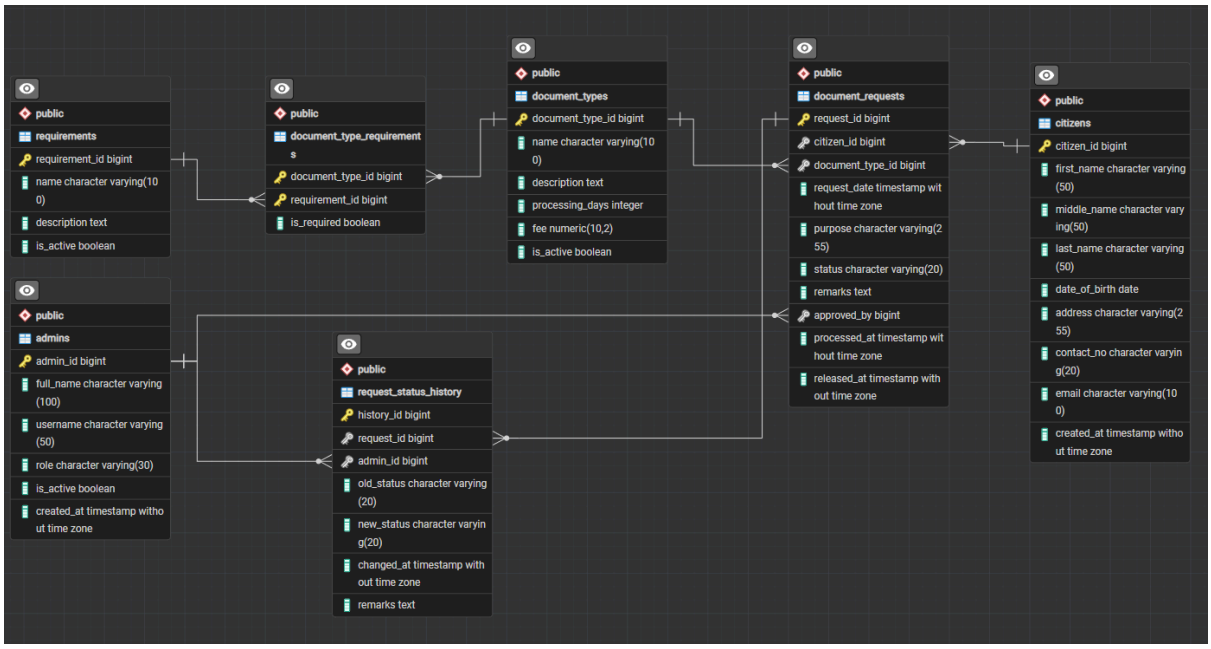


Figure 1. Entity Relationship Diagram of the iBarangay Database.

4.2 Database Schema Summary

Table Name	Description	Primary Key	Foreign Key(s)
citizens	Stores resident demographic and identity information.	citizen_id	None
admins	Stores system administrators and barangay staff accounts.	admin_id	None
document_types	Stores issued document categories, processing times, and fees.	document_type_id	None
requirements	Stores master listing of prerequisites needed for requests.	requirement_id	None
document_type_requirements	Associative table resolving M:N relationship between document types and requirements.	(document_type_id, requirement_id)	document_type_id, requirement_id
document_requests	Stores document applications submitted by citizens.	request_id	citizen_id, document_type_id, approved_by
request_status_history	Audit log maintaining historic status transitions for requests.	history_id	request_id, admin_id

---

4.3 SQL Scripts and Migrations

```sql
-- ============================================
-- 1. AUTOMATIC ID GENERATION
-- ============================================

ALTER TABLE public.admins
ALTER COLUMN admin_id
ADD GENERATED BY DEFAULT AS IDENTITY;

ALTER TABLE public.citizens
ALTER COLUMN citizen_id
ADD GENERATED BY DEFAULT AS IDENTITY;

ALTER TABLE public.document_types
ALTER COLUMN document_type_id
ADD GENERATED BY DEFAULT AS IDENTITY;

ALTER TABLE public.requirements
ALTER COLUMN requirement_id
ADD GENERATED BY DEFAULT AS IDENTITY;

ALTER TABLE public.document_requests
ALTER COLUMN request_id
ADD GENERATED BY DEFAULT AS IDENTITY;

ALTER TABLE public.request_status_history
ALTER COLUMN history_id
ADD GENERATED BY DEFAULT AS IDENTITY;
```

```sql
SELECT setval(
            pg_get_serial_sequence('public.admins', 'admin_id')::regclass,
            COALESCE(MAX(admin_id), 1),
            MAX(admin_id) IS NOT NULL
        )
FROM public.admins;

SELECT setval(
            pg_get_serial_sequence('public.citizens', 'citizen_id')::regclass,
            COALESCE(MAX(citizen_id), 1),
            MAX(citizen_id) IS NOT NULL
        )
FROM public.citizens;

SELECT setval(
            pg_get_serial_sequence('public.document_types', 'document_type_id')::regclass,
            COALESCE(MAX(document_type_id), 1),
            MAX(document_type_id) IS NOT NULL
        )
FROM public.document_types;

SELECT setval(
            pg_get_serial_sequence('public.requirements', 'requirement_id')::regclass,
            COALESCE(MAX(requirement_id), 1),
            MAX(requirement_id) IS NOT NULL
        )
FROM public.requirements;

SELECT setval(
            pg_get_serial_sequence('public.document_requests', 'request_id')::regclass,
            COALESCE(MAX(request_id), 1),
            MAX(request_id) IS NOT NULL
        )
FROM public.document_requests;

SELECT setval(
            pg_get_serial_sequence('public.request_status_history', 'history_id')::regclass,
            COALESCE(MAX(history_id), 1),
            MAX(history_id) IS NOT NULL
```

```
        )
FROM public.document_requests;

SELECT setval(
                pg_get_serial_sequence('public.request_status_history', 'history_id')::regclass,
                COALESCE(MAX(history_id), 1),
                MAX(history_id) IS NOT NULL
        )
FROM public.request_status_history;


-- ============================================
-- 3. ALLOW SYSTEM-GENERATED STATUS HISTORY
-- WITHOUT AN ADMINISTRATOR
-- ============================================

ALTER TABLE public.request_status_history
    ALTER COLUMN admin_id DROP NOT NULL;
```

4.4 Seed Data

Seed Data Category	Purpose	Status
Sample Citizen Profiles	Test resident creation, email uniqueness, and lookup routines	Ongoing
Admin & Staff Accounts	Verify administrative authentication and role filtering	Ongoing
Document Types & Fees	Seed document catalog (e.g., Barangay Clearance, Residency Certificate)	Ongoing
Master Requirements	Seed prerequisites (e.g., Valid ID, Proof of Residency)	Ongoing
Document Requests	Test transaction life cycles, status updates, and constraint triggers	Ongoing

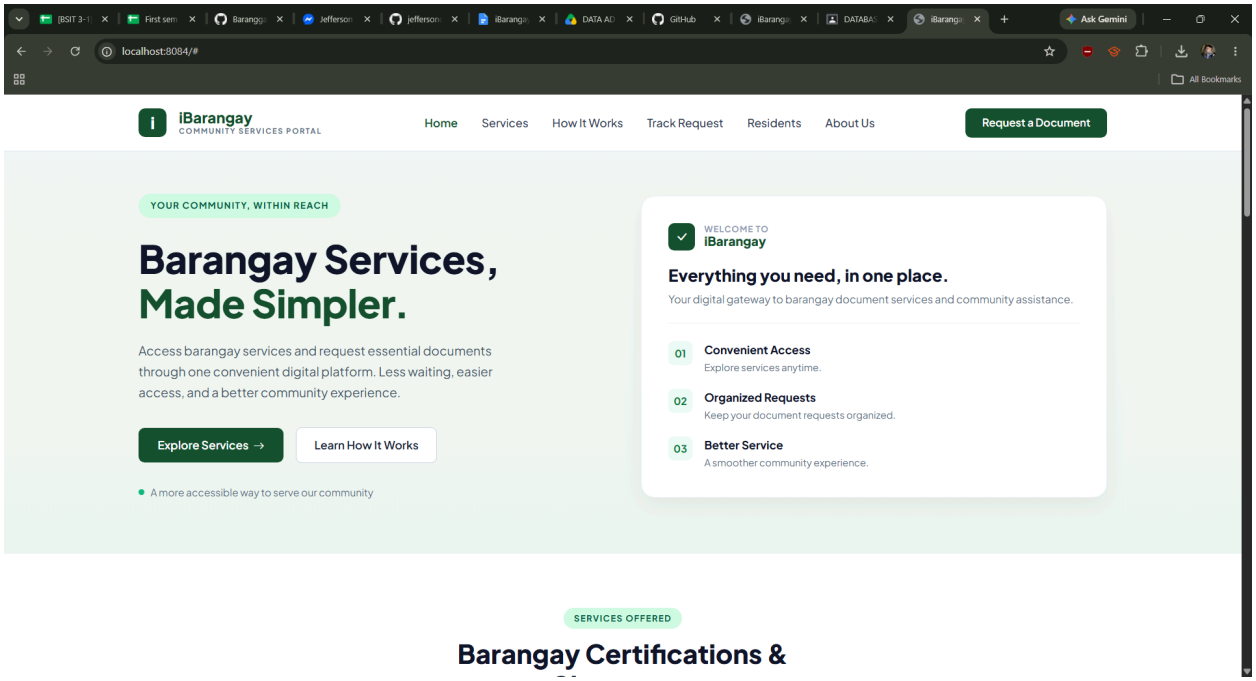
---

5. APPLICATION EVIDENCE

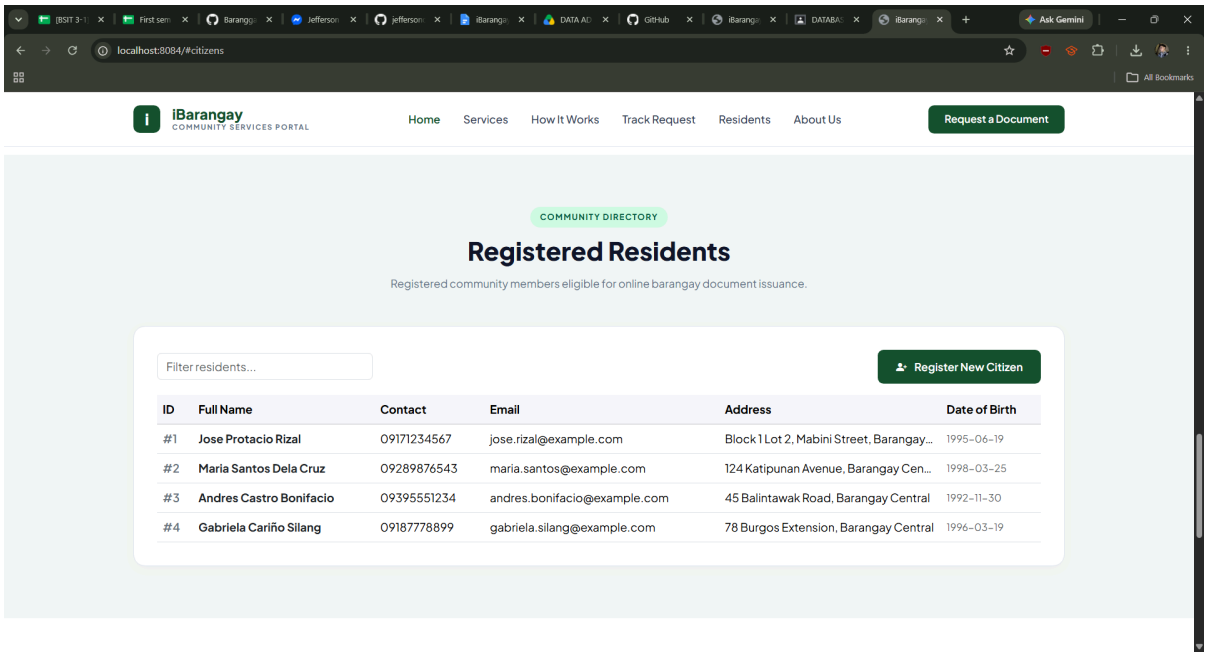
This section will showcase interface screenshots and operational testing logs demonstrating system execution.

5.1 User Interface Evidence

Landing Page Interface



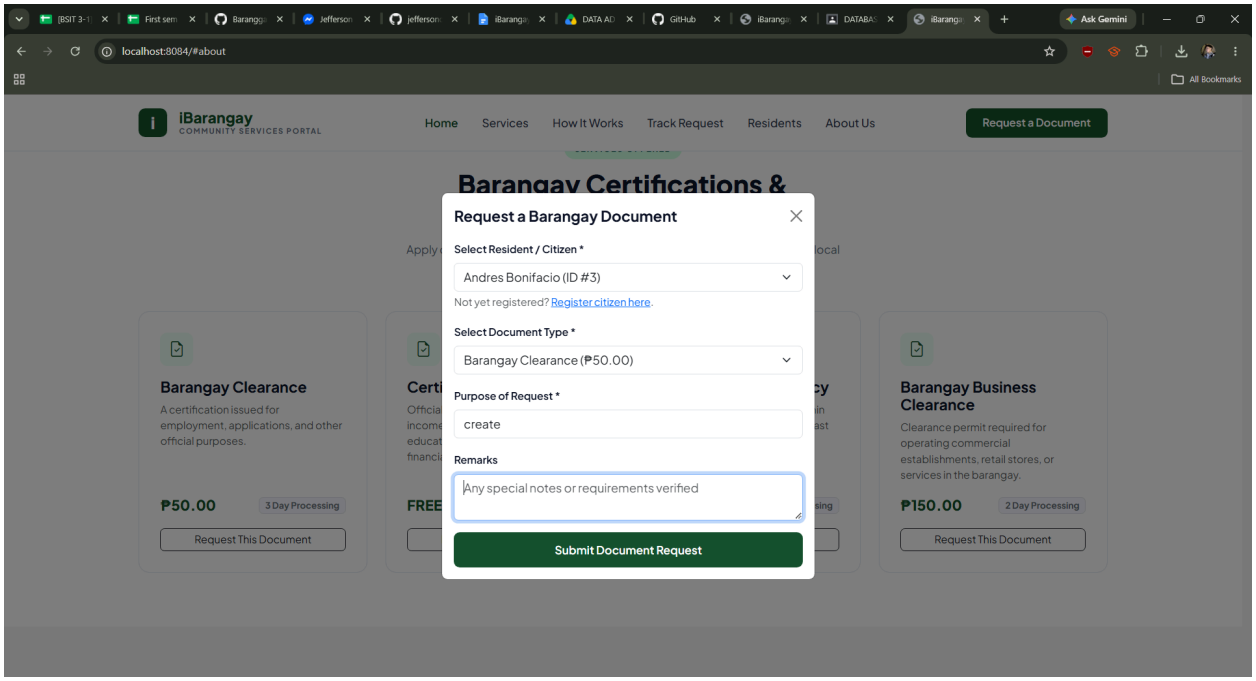
Resident Document Request Module



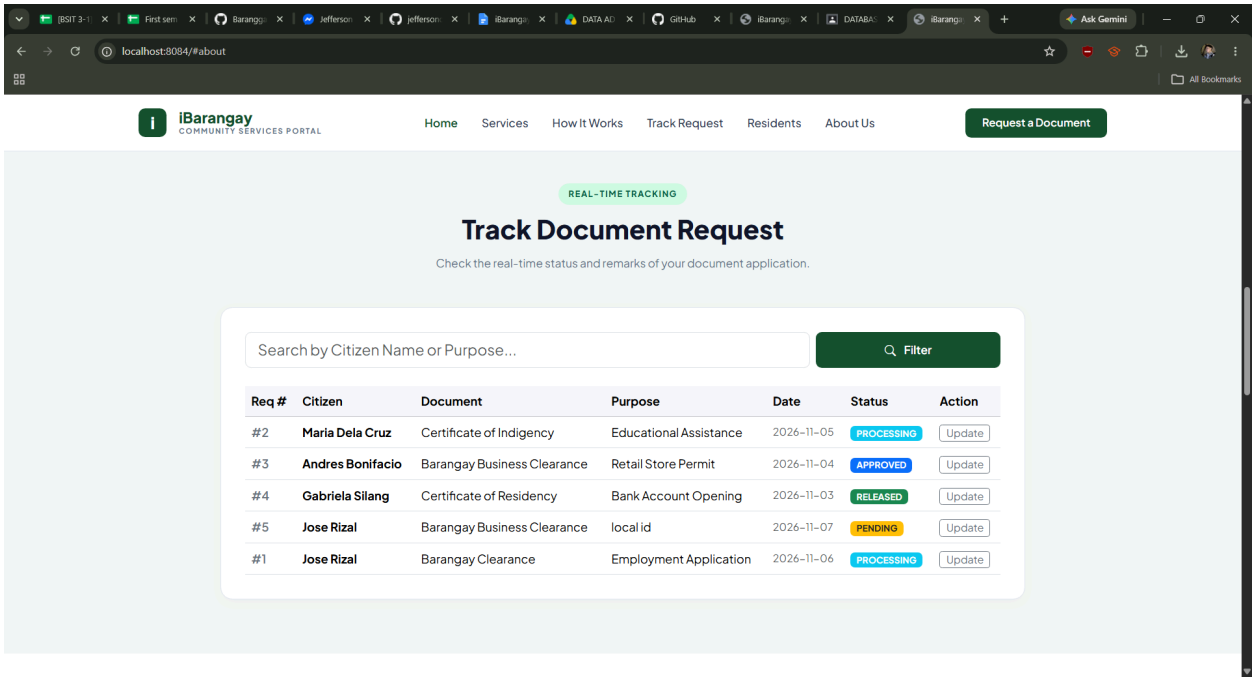
5.2 CRUD Functionality Testing

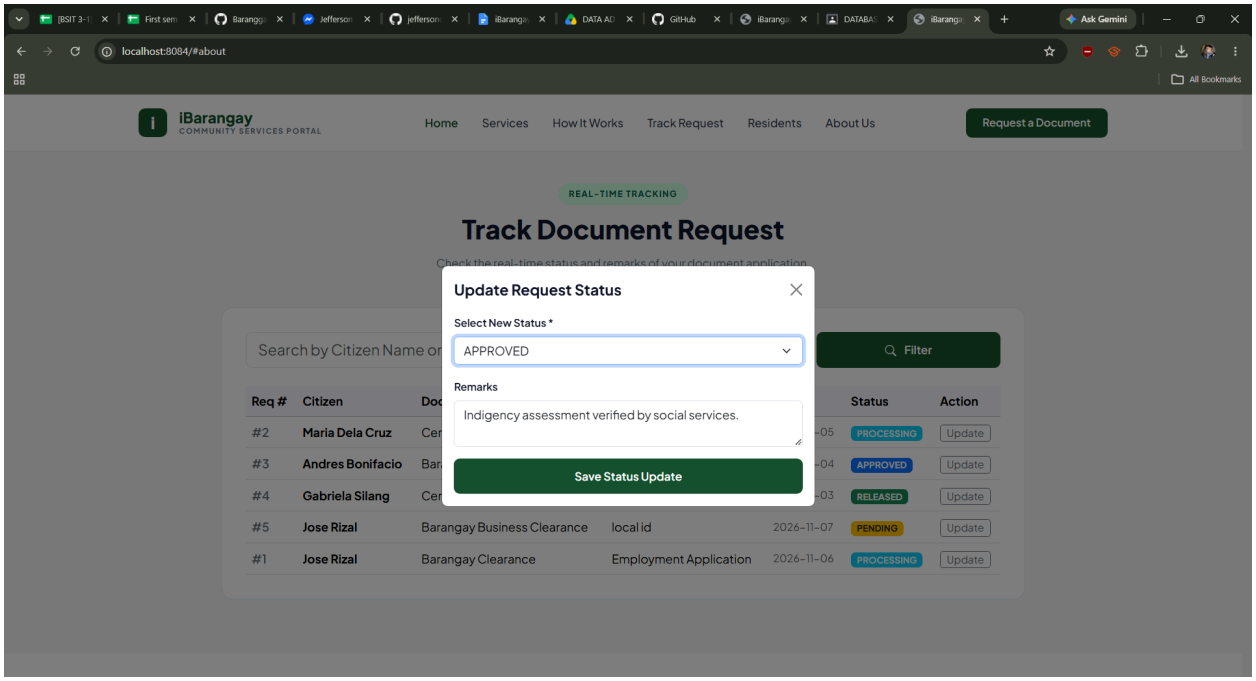
Test Category	Expected Outcome	Status
Create	New resident records and document requests persist accurately in PostgreSQL.	Completed
Retrieve	System queries and filters active requests and resident records without data loss.	Completed
Update	Modifying request status inserts audit entries into status history and updates timestamps.	Completed
Delete	Soft-deleting/deactivating document types or removing pending requests functions properly.	Completed

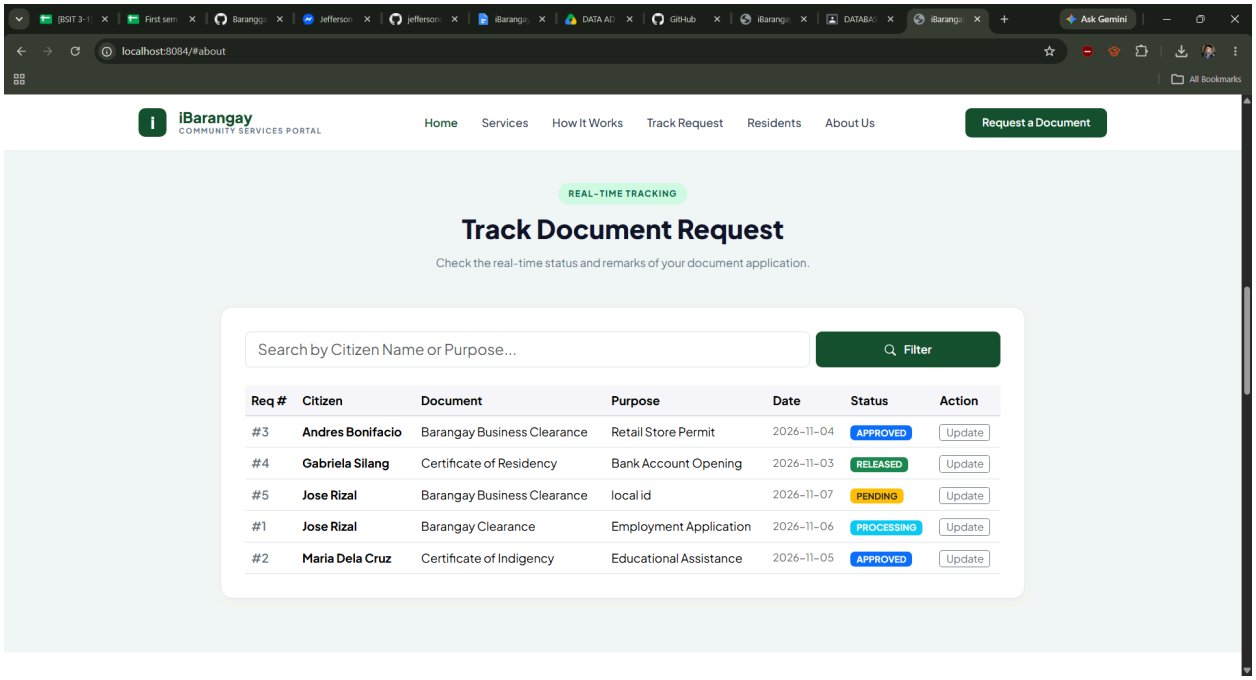
5.2.1 Create Test



5.2.3 Update Test







---

6. SETUP AND CONFIGURATION

6.1 Prerequisites

To run the iBarangay application locally, the following tools must be installed:

- **Spring boot** 4.1.2
- **Git**
- **PostgreSQL Server** 9.17
- **pgAdmin 4**
- Preferred Code Editor (VS Code), IntelliJ IDEA 2026.3

6.2 Installation Steps

1. Clone the project repository using Git.
2. Open the project folder in your development environment.
3. Install necessary backend and frontend dependencies using package managers (npm install).
4. Ensure the PostgreSQL service is running on port 5432.
5. Access PostgreSQL via psql or pgAdmin and create the project database: CREATE DATABASE **PPGG_projectdb**;
6. Connect to grp_ibarangaydb and execute the schema DDL script (schema.sql) to instantiate all 7 tables and constraints.
7. Populate seed data using the initial migration script.
8. Configure environment credentials in .env.

9. Start the application development server (npm run dev).
10. Open the browser and navigate to http://localhost:3000.

Repository: https://github.com/helixfrancisco92-eng/Baranggay-Application

6.3 Environment Variables

Variable	Description	Example Placeholder
DB_HOST	Database server address	[DB_HOST]
DB_USER	Database username	[DB_USER]
DB_PASSWORD	Database password	[DB_PASSWORD]
DB_NAME	Database identifier	[DB_NAME]
DB_PORT	Database server port	[DB_PORT]
PORT	Application server port	[PORT]

6.4 Database User and Role Setup

PostgreSQL database accounts must be established with appropriate grant privileges to safeguard data assets:

- Create a dedicated database service role for the backend application.
- Assign explicit **SELECT**, **INSERT**, **UPDATE**, and **DELETE** permissions to table objects.
- Restrict direct schema alteration (**DROP**, **ALTER**) rights from basic execution roles.
- Verify application connection pools through encrypted connection parameters.

---

7. ISSUES AND RESOLUTIONS

During project development, technical challenges were logged and addressed as shown below:

Problems Encountered	Solutions Applied
Multi-to-many relationship between document types and requirements caused redundant data entries.	Designed an associative table (**document_type_requirements**) with composite primary keys to normalize data to 2NF/3NF.

Problems Encountered	Solutions Applied
Rejection of document requests occurred without mandatory remarks, violating business intent.	Introduced a PostgreSQL **CHECK** constraint (**chk_rejected_has_remarks**) ensuring **status <> 'REJECTED' OR remarks IS NOT NULL**.
The audit trail of status progression was lost when overwriting the status field directly in request tables.	Segregated historical tracking into a dedicated **request_status_history** table populated upon every state transition.
Environment path issues prevented **psql** command execution in Windows terminal.	Added PostgreSQL binary path (**C:\Program Files\PostgreSQL\18\bin**) to system Environment Variables.

7.1 Current Limitations

1. **Lack of Payment Gateway Integration:** Online fee collection currently relies on manual payment verification upon pickup rather than third-party payment APIs (e.g., GCash API).
2. **SMS Notifications:** Automated SMS alerts for ready-for-pickup notices are not yet integrated; notification is confined to the user web portal.
3. **External Government Sync:** The system operates independently within the barangay and does not sync directly with city-level or national databases.
4. **Mobile App Optimization:** The web interface is responsive, but a dedicated native mobile application has not been generated.

---

8. AI USAGE DECLARATION

This section declares the use of artificial intelligence tools during the preparation and development of the iBarangay project.

8.1 AI Tools Used

AI Tool	Purpose of Use
ChatGPT	Assistance in structuring ERD relationships, organizing DDL constraint syntax, and drafting text content.
Gemini	Assistance in refining database normalization explanations, validating business rules, and technical editing.

AI Tool	Purpose of Use
Grammarly	Editing assistance to maintain grammatical correctness across project documentation.

8.2 Tasks Assisted by AI

- Generating initial database table templates and constraint validation ideas.
- Assisting in formatting complex Markdown data dictionaries and schema outlines.
- Refining project narrative tone and verifying 3rd person documentation style.

8.3 Validation and Verification

All AI-assisted suggestions were thoroughly analyzed, tested, and validated by the team prior to inclusion. The developers executed and verified all generated DDL scripts inside PostgreSQL to confirm complete schema compatibility. The team takes full responsibility for the architecture, business logic, codebase, and documentation submitted.